Getting Started with Raspberry Pi Board and IMU Sensor Interfacing

Alen Smailov
Robotics Engineering, SEDS
Nazarbayev University
Astana, Kazakhstan
alen.smailov@nu.edu.kz

Ilyas Abutalip
Robotics Engineering, SEDS
Nazarbayev University
Astana, Kazakhstan
ilyas.abutalip@nu.edu.kz

Ismail Mamadayev
Robotics Engineering, SEDS
Nazarbayev University
Astana, Kazakhstan
ismail.mamadayev@nu.edu.kz

***Abstract*—This report details the initial setup of a Raspberry Pi 4 Model B microcomputer and its integration with a Pololu MinIMU-9 v5 inertial measurement unit. The project covers OS installation, network configuration via SSH, version control using Git and I2C sensor communication protocols.**

I. Introduction

The Raspberry Pi is a compact single-board microcomputer integrating a multi-core ARM processor and GPIO pins. It serves as a platform for developing embedded Linux applications and interacting with sensors. This project utilizes the RPi 4 Model B to interface with a Pololu MinIMU-9 v5 sensor via I2C.

II. Task 1: Getting Started with Raspberry Pi

A. OS Configuration and Safety

The Raspberry Pi OS (64-bit) was flashed onto a microSD card. To ensure hardware longevity strict safety protocols were followed:

- The board was never placed on metal surfaces.
- Peripherals (HDMI, USB) were connected before power-up.
- System shutdown was performed via `sudo shutdown -h now` to prevent OS image corruption.

B. Networking and Remote Access

Remote access was established using SSH from a virtual Ubuntu OS. As shown in Fig. 1, the connection was initiated using the board's IP address.

III. Task 2: Git Practice

Version control was managed through GitHub Classroom as required for centralized submission. The project repository, containing the source code and PDF report, can be accessed at: `https://github.com/ISMAILMamadayev/ICE_Lab1`

```
#include <iostream>
int main() {
    std::cout << "Hello World" << std::endl;
    return 0;
}
```

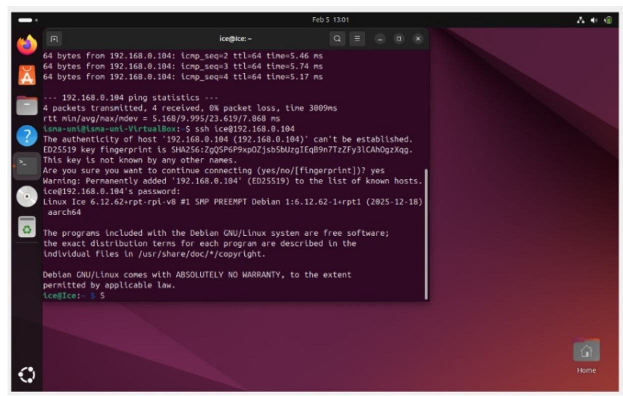

Fig. 1. Successful SSH remote connection to the Raspberry Pi at 192.168.0.104.

IV. Task 3: IMU Sensor Interfacing

A. Hardware Connection

The Pololu MinIMU-9 v5 was connected to the RPi physical pins as follows:

- **VDD** to Pin 1 (3.3V).
- **SDA** to Pin 3 (GPIO 2).
- **SCL** to Pin 5 (GPIO 3).
- **GND** to Pin 6.

Note: VIN was left unconnected because the RPi provides a regulated 3.3V to VDD, and the GPIO pins are not 5V tolerant[cite: 171].

B. I2C Communication and Configuration

To meet the project requirements of 26Hz for the Gyro/Accel and 5Hz for the Magnetometer[cite: 191], specific control words were written to the configuration registers.

The value `0x20` in `CTRL1_XL` sets the Output Data Rate (ODR) to 26Hz. The command `i2cget -y 1` uses the value `1` to specify the I2C bus number on the Raspberry Pi 4.

V. Results and Analysis

Real-time data was observed using the `watch` command to monitor the High Byte of the accelerometer:

```
watch -n 0.1 i2cget -y 1 0x6b 0x29
```

TABLE I
SENSOR CONFIGURATION AND DATA REGISTERS

Sensor	Register	Value	Function
LSM6DS33	CTRL1_XL (0x10)	0x20	Accel: 26Hz, $\pm$2g
LSM6DS33	CTRL2_G (0x11)	0x24	Gyro: 26Hz, $\pm$500 dps
LIS3MDL	CTRL_REG1 (0x20)	0x10	Mag: 5Hz, Low Power
LIS3MDL	CTRL_REG2 (0x21)	0x20	Mag: $\pm$4 gauss range
LSM6DS33	OUTX_L_G (0x22)	Read-only	Gyro X-axis Low Byte
LSM6DS33	OUTX_H_G (0x23)	Read-only	Gyro X-axis High Byte

This allowed for the verification of measurement changes during physical rotation of the sensor.

VI. INDIVIDUAL CONTRIBUTIONS

In accordance with the project requirements, the individual contributions of the team members are outlined below:

- **Alen Smailov**: Primarily responsible for Task 3, including the physical wiring of the MinIMU-9 v5 sensor and determining the I2C control words. Contributed significantly to the technical documentation in the final report.
- **Ilyas Abutalip**: Focused on Task 3 sensor interfacing, testing the `i2c-tools` functionality and verifying measurement data. Assisted in the drafting and formatting of the report.
- **Ismail Mamadayev**: Responsible for Task 1 and Task 2. This included the initial Raspberry Pi OS configuration, networking setup via SSH and managing the GitHub repository, version control and initial LaTeX project structure.

VII. CONCLUSION

The successful configuration of the Raspberry Pi and the retrieval of IMU data via I2C demonstrates a foundational understanding of mechatronic system design and embedded Linux environments.